

ONLINE QUIZ ASSESSMENT PLATFORM: STUDENT PRESENTATION NOTES

1. FRONTEND LEAD & UX DESIGNER (STUDENT 4)

- **Our platform provides a clean, intuitive interface for both candidates and instructors, focusing on ease of use.**
- **I'll quickly tour the main dashboards and the responsive design built using React and Tailwind CSS.**
- **We manage complex state and data flow efficiently using Redux Toolkit.**
- **We prioritized accessibility features and clear user guidance throughout the application.**

Transition Cue: "Now, [Student 2's Name] will walk us through the quiz interaction and client-side logic."

2. FRONTEND DEVELOPER & CLIENT LOGIC (STUDENT X)

- **The Quiz Player is dynamic and robust, built to handle timeboxing and question randomization.**
- **I'll demonstrate the Result View and the secure process for submitting answers.**
- **Crucially, we integrate client-side anti-cheating by monitoring browser focus and keystroke/tab-switch events which feeds directly into the AI system.**
- **We use route guards and client-side logic to ensure students are properly authenticated and authorized before accessing the exam.**

Transition Cue: "To understand how this data is secured and processed, [Student 3's Name] will explain the API and core security layer."

3. BACKEND LEAD & API SPECIALIST (STUDENT 2)

- **Our system runs on a highly scalable, containerized MERN architecture (MongoDB, Express, React, Node.js).**

- The backend is an Express API defining clear routes for quizzes, results, and proctoring events.
- Security starts with our JWT access/refresh token system and optional 2FA routes.
- We manage traffic and abuse using rate limiting middleware and role-based route guards.
- Our core engine supports advanced features like adaptive difficulty via `Quiz.getNextAdaptiveQuestion()`.

Transition Cue: "The crucial security layer, our AI proctoring, and real-time data handling will be detailed by [Student 4's Name]."

4. AI PROCTORING & DATA ENGINEER (STUDENT 3)

- Our anti-cheating measures rely on a multi-signal approach: face presence, gaze deviation, and browser focus.
- We use Socket.io to provide a secure, real-time channel for proctoring alerts back to the instructor dashboard.
- All events are timestamped and assigned a severity before being logged to the `ProctoringEvent` collection for manual review.
- Regarding privacy, we encrypt credentials and isolate biometric templates in separate, secured collections.
- We use MongoDB schemas specifically designed for high-volume quiz, result, and audit log data.

Transition Cue: "Finally, [Student 5's Name] will wrap up by covering the reliability, deployment, and testing that makes this system ready for university-wide use."

5. DEVOPS & INFRASTRUCTURE COORDINATOR (STUDENT 5)

- We use Docker Compose for local orchestration, ensuring our environment is consistent from development to deployment.
- The architecture is inherently scalable; services are stateless, supporting simple horizontal scaling.
- Our reliability is backed by a strong testing strategy—Jest/Supertest on the backend and Vitest/Testing Library on the frontend.
- We manage risks through audit logs, robust error handling, and documented rollback plans.

- Our future roadmap includes pose estimation, LMS/SIS connectors, and federated learning.

Conclusion: "This platform is a comprehensive, secure, and scalable solution ready to redefine online assessment."



GROUP COORDINATION NOTES

- **Time Check:** Target 4 minutes per speaker (20 minutes total) plus 5 minutes for Q&A.
- **Slide Deck:** Use the cohesive deck. Point to key areas on the Architecture Diagram and Dashboards.
- **Live Demo:** Be ready for the demo. Start the Docker containers before the presentation begins.
- **Handover:** Listen to the previous student's final point to ensure your transition is seamless.